Biomed Data School - Terminal for Bioinformatics Cheat Sheet

Mental model: project folder -> download -> inspect -> QC -> analyse -> log -> script.

Before pressing Enter: where am I, what are inputs, what changes and where do outputs/logs go?

1. Clean project folder

Command	Use
`raw_data/`	Original files; keep read-only.
`metadata/`	Sample sheets and phenotypes.
`references/`	Genome, GTF/GFF and indexes.
`results/ ; logs/`	Outputs and command logs.
`mkdir -p project/{raw_data,data,metadata,references,results,scripts,logs} ; cd project ; pwd ; ls -lh`	Set up project structure and check contents.

Rule: never analyse real biomedical data in Downloads or a random HPC folder.

2. Move safely

Command	Use
`pwd ; ls -lh ; cd ..`	Know where you are and what is there.
`mkdir -p folder/`	Create folders.
`cp file backup.txt ; mv old new`	Copy or rename/move files.
`rm -i file`	Delete carefully; no recycle bin.
`nano script.sh ; vim script.sh`	Edit files in terminal.
`chmod +x script.sh`	Make a script executable.

3. Inspect before analysis

Command	Use
`file sample.fastq.gz`	Guess file type.
`du -sh * ; df -h`	File sizes and free disk.
`head file ; tail file ; less -S file`	Preview tables or text.
`wc -l file.tsv`	Count lines.
`zcat file.gz ; zless file.gz ; zgrep "pattern" file.gz`	Inspect compressed text.

4. Search tables

Command	Use
`grep -i "brca1" genes.tsv`	Search text.
`cut -f1,3 metadata.tsv`	Select TSV columns.
`sort file | uniq -c`	Count unique values.
`awk -F'\t' 'NR==1 || $3=="case"' metadata.tsv`	Header + filtered rows.
`tr -d "\r" < in.tsv > out.tsv`	Remove Windows line endings.
`cut -f3 metadata.tsv | sort | uniq -c | sort -nr`	Count unique and sort.

5. Connect commands and save logs

Command	Use
`cmd1 | cmd2`	Pipe output into next command.
`command > out.txt`	Save output.
`command 2> err.log`	Save errors.
`command 2>&1 | tee run.log`	See output and save log.
`history > logs/commands.txt`	Save command history.

6. FASTQ / FASTA essentials

Command	Use
`zcat R1.fastq.gz | head -n 12`	Inspect first 3 FASTQ records.
`zcat R1.fastq.gz | wc -l`	Approximate reads = lines / 4.
`seqkit stats *.fq.gz`	Robust sequence summary.
`seqkit sample -n 10000 reads.fq.gz > subset.fq`	Create small subset.
`samtools faidx reference.fa`	Index FASTA.

Core file types: FASTQ reads, FASTA reference, GTF/GFF annotation, BAM alignment and VCF variants.

7. Download and verify

Command	Use
`wget -c URL`	Download and resume.
`curl -L -o file URL`	Download with redirects.
`wget -O name URL ; wget --timestamping URL`	Choose filename or refresh only if newer.
`rsync -avP source/ target/`	Robust transfer.
`md5sum file ; sha256sum file`	Check integrity.

8. SRA Toolkit

```
prefetch SRR123456 2>&1 | tee logs/prefetch.log
fasterq-dump SRR123456 --split-files -O raw_data \
2>&1 | tee logs/fasterq.log
gzip raw_data/*.fastq
seqkit stats raw_data/*.fastq.gz
```

Teaching tip: keep a tiny backup FASTQ locally. Live SRA can fail because of cache, quota or network issues.

9. QC first win

```
fastqc raw_data/*.fastq.gz -o results/fastqc \
2>&1 | tee logs/fastqc.log
multiqc results/fastqc -o results/multiqc \
2>&1 | tee logs/multiqc.log
```

Ask: Which sample looks different? Adapter content? Read counts? Low-quality tails?

10. Batch many files

Command	Use
`echo raw_data/*.fastq.gz`	Preview wildcard expansion.
`find . -name "*.bam"`	Find files recursively.
`find raw_data -name "*.fq.gz" | xargs -n 1 basename`	Pass many files onward.
`parallel -j 4 fastqc {} ::: raw_data/*.fq.gz`	Optional parallel batch run.

`for f in raw_data/*.fastq.gz; do`
`sample=$(basename "$f" .fastq.gz)`
`echo "$sample"`
`done`
	Print sample names.

11. BAM, VCF and counts

Command	Use
`samtools view -H sample.bam ; samtools quickcheck sample.bam`	Inspect header and check integrity.
`samtools flagstat sample.bam`	Mapping summary.
`samtools sort in.bam -o sorted.bam`	Sort alignments.
`samtools index sorted.bam`	Index BAM.
`bcftools view variants.vcf.gz | head`	Inspect variants.
`bcftools stats variants.vcf.gz`	Variant statistics.
`bedtools intersect -a A.bed -b B.bed`	Overlap intervals.
`samtools depth sorted.bam > depth.tsv`	Per-base coverage.

12. Reproducibility and HPC

Command	Use
`set -euo pipefail`	Make scripts fail loudly.
`which fastqc ; fastqc --version`	Record exact software.
`conda env list ; conda activate rnaseq`	Choose a software environment.
`ssh user@server`	Connect to server.
`tmux new -s qc`	Keep long work alive.
`module avail ; module load fastqc`	Find and load HPC modules.
`squeue -u $USER ; scancel JOBID`	Check or cancel jobs.
`module load fastqc ; sbatch script.sh`	Typical HPC usage.

Final mantra: inspect, transform, document, save logs and leave a folder another scientist can audit.